Chapter 2. Google ADK 개발 환경 구축 및 첫 번째 Agent 실행

학습 목표

이번 장에서는 Google ADK 개발 환경을 구성하고 첫 번째 AI Agent를 실행합니다.

학습이 끝나면 다음 내용을 수행할 수 있습니다.

- Google ADK 개발 환경 구축
- Python 가상환경 생성
- API Key 설정
- 기본 Agent 프로젝트 생성
- 첫 번째 Agent 실행

이 장의 용어

개발 환경을 다루는 장이라 도구 이름이 많이 나옵니다. 각 도구가 "무엇을 하는 물건인지"만 알고 있으면 절차를 따라갈 수 있습니다.

용어	영문 · 약어 풀이	설명
Google ADK	Agent Development Kit, 에이전트 개발 도구 모음	Google이 제공하는 Python 기반 AI Agent 개발 프레임워크
프레임워크	Framework, 뼈대	자주 쓰는 기능을 미리 만들어 둔 개발 뼈대. 개발자는 그 위에 필요한 부분만 작성
라이브러리	Library	특정 기능을 묶어 둔 코드 모음. 필요할 때 가져다 씀. python-dotenv, requests 등
패키지	Package	라이브러리를 설치·배포하는 단위. pip로 설치
pip	Pip Installs Packages, Python 패키지 설치 도구	`pip install 이름` 으로 라이브러리를 내려받아 설치하는 명령
의존성	Dependency	어떤 패키지가 동작하기 위해 함께 필요한 다른 패키지. pip가 자동으로 함께 설치
가상환경	venv, Virtual Environment	프로젝트마다 독립된 Python 실행 환경. 라이브러리 버전 충돌을 막음

용어	영문 · 약어 풀이	설명
터미널	Terminal, 명령 프롬프트(cmd), PowerShell	글자로 명령을 입력해 컴퓨터를 조작하는 창. VS Code 안에도 내장되어 있음. 이 과정은 Windows 명령 프롬프트(cmd) 기준
VS Code	Visual Studio Code	Microsoft의 무료 코드 편집기. 코드 작성·실행·터미널을 한곳에서 처리
PATH	환경 변수 PATH	터미널이 명령어 파일을 찾는 폴더 목록. 여기에 등록되어야 `python`, `code` 같은 명령이 동작
Git / GitHub	—	Git은 코드 변경 이력을 관리하는 도구, GitHub은 Git 저장소를 올려 공유하는 서비스
.gitignore	—	Git에 올리지 않을 파일 목록을 적는 파일. .env를 여기에 추가
API Key	API 키, 인증 키	서비스가 "누가 호출했는지"를 확인하는 비밀 문자열. 비밀번호처럼 다룸
환경 변수	Environment Variable	프로그램 바깥(운영체제 또는 .env 파일)에 저장해 두고 코드에서 읽어 오는 설정값
.env	dotenv 파일	환경 변수를 `이름=값` 형태로 적어 두는 파일. python-dotenv 라이브러리가 읽음
requirements.txt	의존성 목록 파일	프로젝트에 필요한 패키지 이름과 버전을 적은 파일. 다른 PC에서 같은 환경을 재현할 때 사용
CLI	Command Line Interface	터미널에서 명령어로 사용하는 프로그램 방식. `adk web`, `adk run`이 ADK의 CLI
localhost / 127.0.0.1	로컬호스트	"내 컴퓨터 자신"을 가리키는 주소. 뒤의 `:8000`은 그 안에서 프로그램을 구분하는 포트(Port) 번호
Gemini	—	Google의 LLM 제품군. Flash(빠르고 저렴), Pro(고성능) 등 등급이 있음
Instruction	인스트럭션, 지시문	Agent의 역할과 규칙을 정의하는 설정. System Prompt와 같은 역할. Chapter 6에서 자세히 다룸
Multi Agent	멀티 에이전트	역할이 다른 여러 Agent가 협력하는 구조. 이 과정에서는 개념만 소개

1. Google ADK 소개

1.1. Google ADK란?

앞 장에서는 AI Agent의 개념과 구성 요소를 학습했습니다.

AI Agent는 단순한 챗봇이 아니라 **목표를 수행하기 위해 LLM과 Tool을 활용하는 시스템**이라는 것을 알게 되었습니다.

그렇다면 이러한 AI Agent는 어떻게 개발할 수 있을까요?

예를 들어 Python만 이용하여 AI Agent를 구현한다고 가정해 보겠습니다.

개발자는 다음과 같은 기능을 직접 구현해야 합니다.

- LLM 호출
- 프롬프트 작성
- Tool 관리
- Tool Calling 처리
- 대화(Context) 관리
- Session 관리
- 여러 Agent 간 협업(Multi Agent)
- 오류 처리
- Workflow 구성

이 모든 기능을 직접 구현하려면 많은 시간과 노력이 필요합니다.

이러한 문제를 해결하기 위해 등장한 것이 **Google ADK(Agent Development Kit)** 입니다.

Google ADK는 Google에서 제공하는 **AI Agent 개발 프레임워크(Framework)**로, Agent 개발에 필요한 다양한 기능을 미리 제공하여 개발자가 핵심 로직에 집중할 수 있도록 도와줍니다.

즉,

> **Google ADK는 AI Agent를 보다 쉽고 빠르게 개발하기 위한 Python 기반 프레임워크입니다.**

1.2. 왜 Google ADK가 필요한가?

간단한 AI 프로그램이라면 LLM API를 직접 호출해도 충분할 수 있습니다.

예를 들어 사용자의 질문을 받아 Gemini API를 호출하고 결과를 출력하는 프로그램은 몇 줄의 코드만으로도 작성할 수 있습니다.

하지만 실제 서비스는 훨씬 복잡합니다.

예를 들어 다음과 같은 요구사항을 생각해 보겠습니다.

> "사용자가 '홍길동의 정보를 조회하고 이메일로 보내줘.'라고 요청하면, 데이터베이스에서 홍길동의 정보를 조회한 후 이메일을 발송한다."

이 작업을 직접 구현하려면 다음과 같은 과정이 필요합니다.

```
사용자 요청

↓

LLM 호출

↓

Tool 필요 여부 판단

↓

Database Tool 실행

↓

조회 결과 확인

↓

Gmail Tool 실행

↓

결과 생성

↓

사용자 응답
```

이러한 흐름을 모두 직접 작성하는 것은 쉽지 않습니다.

Google ADK는 이러한 복잡한 과정을 프레임워크가 처리해 주므로, 개발자는 Agent와 Tool만 정의하면 됩니다.

1.3. Google ADK의 역할

Google ADK는 AI Agent 개발을 위한 다양한 기능을 제공합니다.

대표적인 기능은 다음과 같습니다.

- Agent 생성
- Tool 등록 및 실행
- Tool Calling
- Multi Agent 구성
- Session 관리
- Context 관리
- Workflow 구성

개발자는 이러한 기능을 직접 구현하지 않고도 사용할 수 있습니다.

1.4. Google ADK의 주요 기능

① Agent 생성

Google ADK에서는 `Agent` 객체를 생성하여 AI Agent를 정의합니다.

Agent에는 다음과 같은 정보를 설정합니다.

- Agent 이름
- 사용할 LLM
- 역할(Instruction)
- 사용할 Tool

예를 들어 하나의 Agent를 생성하면

```
사용자

↓

Agent

↓

응답
```

과 같은 기본 구조를 쉽게 만들 수 있습니다.

---

② Tool Calling

Google ADK는 Tool Calling을 기본적으로 지원합니다.

예를 들어 사용자가

> "현재 시간을 알려줘."

라고 질문하면

Agent는 자동으로 시간 조회 Tool을 실행합니다.

개발자가 직접

```
get_current_time()
```

를 호출하는 코드를 작성할 필요가 없습니다.

Google ADK가 내부적으로 Tool을 선택하고 실행합니다.

---

③ Multi Agent

실제 서비스에서는 하나의 Agent보다 여러 Agent가 협력하는 경우가 많습니다.

예를 들어

```
Supervisor Agent
├── 여행 Agent
├── 금융 Agent
├── 개발 Agent
└── 고객센터 Agent
```

Google ADK는 이러한 Multi Agent 구조를 쉽게 구현할 수 있도록 지원합니다.

---

④ Context 관리

AI Agent는 이전 대화를 기억해야 자연스러운 대화를 이어갈 수 있습니다.

Google ADK는 Session과 Context를 관리하여 연속적인 대화를 지원합니다.

예를 들어

```
오늘 서울 날씨 알려줘.
↓
내일은?
```

```
↓
모레는?
```

와 같은 대화를 자연스럽게 처리할 수 있습니다.

---

⑤ Workflow 구성

Google ADK에서는 여러 Tool을 연결하여 하나의 Workflow를 구성할 수 있습니다.

예를 들어

```
사용자 요청
↓
문서 읽기
↓
AI 요약
↓
이메일 발송
↓
완료
```

와 같은 작업을 하나의 Agent 안에서 수행할 수 있습니다.

1.5. Google ADK의 특징

Google ADK는 다음과 같은 특징을 가지고 있습니다.

Python 기반

Google ADK는 Python으로 개발됩니다.

Python 함수와 클래스를 그대로 활용할 수 있으며, 기존 Python 라이브러리와도 쉽게 연동됩니다.

Gemini 모델 지원

Google ADK는 Google의 Gemini 모델을 기본적으로 지원합니다.

Gemini Flash, Gemini Pro 등 다양한 모델을 사용할 수 있으며, 모델 변경도 간단합니다.

Tool Calling 지원

Python 함수를 Tool로 등록하면 Agent가 필요할 때 자동으로 호출합니다.

이를 통해 외부 API, 데이터베이스, 파일 시스템 등 다양한 기능을 Agent와 연결할 수 있습니다.

Multi Agent 지원

여러 Agent를 역할별로 분리하여 협업하는 구조를 쉽게 구현할 수 있습니다.

이를 통해 대규모 AI 서비스를 효율적으로 설계할 수 있습니다.

높은 확장성

Google ADK는 다양한 Python 라이브러리 및 외부 서비스와 연동할 수 있습니다.

예를 들어

- MySQL
- PostgreSQL
- REST API
- Google Drive
- MCP Server

등을 Agent와 쉽게 연결할 수 있습니다.

1.6. Google ADK 개발 흐름

Google ADK를 이용한 AI Agent의 기본적인 동작 과정은 다음과 같습니다.

```
사용자

↓

Agent

↓

Gemini

↓
```

```
Tool (필요한 경우)

↓

결과 생성

↓

사용자
```

보다 자세히 표현하면 다음과 같습니다.

```
① 사용자가 자연어로 요청

↓

② Agent가 요청을 전달

↓

③ Gemini가 질문을 분석

↓

④ Tool이 필요한지 판단

↓

⑤ 필요한 Tool 실행

↓

⑥ Tool 결과 반환

↓

⑦ Gemini가 최종 응답 생성

↓

⑧ 사용자에게 결과 전달
```

이 과정에서 개발자는 Tool을 직접 호출하지 않습니다.

Agent가 Gemini와 협력하여 적절한 Tool을 선택하고 실행합니다.

1.7. Google ADK를 사용하는 이유

Google ADK를 사용하면 다음과 같은 장점이 있습니다.

직접 구현	Google ADK
LLM 호출 코드 작성	Agent가 자동 처리
Tool 관리 직접 구현	Tool 등록만으로 사용 가능
Context 관리 구현	Session 기반 자동 관리
Multi Agent 직접 구현	프레임워크에서 지원
Workflow 직접 구성	Agent 중심으로 간단하게 구현

즉, Google ADK는 반복적으로 작성해야 하는 공통 기능을 프레임워크가 대신 처리해 주므로 개발자는 **비즈니스 로직과 Agent 설계에 집중**할 수 있습니다.

정리

이번 절에서는 Google ADK의 개념과 주요 기능을 살펴보았습니다.

개념	설명
Google ADK	AI Agent 개발을 위한 Python 프레임워크
Agent	AI의 역할과 동작을 정의하는 객체
Tool Calling	AI가 Tool을 자동으로 선택하고 실행
Multi Agent	여러 Agent가 협업하는 구조
Context	이전 대화를 활용한 연속적인 대화 지원
Workflow	여러 작업을 연결하여 하나의 업무 수행

---

2. 개발 환경 준비

2.1. Python 설치

Python이 설치되어 있지 않다면 공식 홈페이지에서 다운로드하여 설치합니다.

공식 홈페이지 : https://www.python.org/

에서 Python 3.12 이상 버전의 Windows 설치 파일을 다운로드해 설치합니다. 최신 버전이 나온 직후에는 라이브러리가 아직 지원하지 않을 수 있으므로, 설치 오류가 나면 한 단계 이전 버전을 선택합니다.

> ⚠️ 설치 화면 첫 페이지 하단의 **Add python.exe to PATH**를 반드시 체크한 뒤 **Install Now**를 누릅니다. 체크하지 않으면 명령 프롬프트에서 `python` 명령을 찾지 못합니다.

설치 이후 시작 메뉴에서 `cmd`를 검색해 **명령 프롬프트**를 새로 열고 아래 명령으로 설치를 확인합니다.

```
python --version
```

> **참고**
>
> `python`을 입력했을 때 Microsoft Store가 열리거나 "내부 또는 외부 명령이 아닙니다"라고 나오면 설치 시 Add python.exe to PATH를 체크하지 않은 것입니다. 이 경우 Python 런처인 `py` 명령을 대신 사용할 수 있습니다(예: `py --version`, `py -m venv venv`). 근본적으로는 Python을 다시 설치하면서 해당 항목을 체크합니다.

정상적으로 설치되었으면 아래와 같이 출력됩니다.

```
Python 3.14.6
```

2.2. Visual Studio Code 설치

Google ADK는 Python 기반 프레임워크이므로 다양한 개발 도구를 사용할 수 있습니다.

이번 교재에서는 **Visual Studio Code(VS Code)** 를 사용하여 모든 실습을 진행합니다.

VS Code는 Microsoft에서 제공하는 무료 코드 편집기로 Python 개발에 필요한 다양한 기능과 확장 프로그램을 제공합니다.

또한 Git 연동, 터미널, 디버깅 기능 등을 기본적으로 지원하여 Google ADK 프로젝트를 개발하기에 매우 적합합니다.

VS Code의 주요 특징

- 무료 사용 가능
- Windows, macOS, Linux 지원
- Python 개발 지원
- Git 연동
- 다양한 확장 프로그램 제공
- 내장 터미널 제공
- 디버깅 기능 지원

2.3. VS Code 다운로드

웹 브라우저를 실행한 후 VS Code 공식 홈페이지에 접속합니다.

```
https://code.visualstudio.com/
```

운영체제에 맞는 설치 파일을 다운로드합니다.

예를 들어

- Windows → Windows Installer (64-bit)
- macOS → Universal
- Linux → Debian / RPM

중 자신의 운영체제에 맞는 버전을 선택하면 됩니다.

2.4. VS Code 설치

다운로드한 설치 파일을 실행합니다.

설치 과정은 일반적인 프로그램 설치와 동일하며 특별한 설정이 없다면 기본 옵션을 그대로 사용해도 됩니다.

Windows에서는 다음 옵션을 선택하는 것을 권장합니다.

```
☑ Add "Open with Code"

☑ Add "Open Folder as Code"

☑ Register Code in PATH
```

특히

```
Register Code in PATH
```

옵션을 선택하면 터미널에서

```
code .
```

명령으로 현재 폴더를 바로 VS Code에서 열 수 있어 매우 편리합니다.

설치가 완료되면 VS Code를 실행합니다.

2.5. 프로젝트(Project)란?

프로젝트(Project)는 하나의 프로그램을 개발하기 위한 작업 공간입니다.

Google ADK 프로젝트에서는 앞으로 작성할 모든 파일이 하나의 프로젝트 폴더 안에서 관리됩니다.

예를 들어 다음과 같은 파일들이 프로젝트 내부에 생성됩니다.

- Python 코드
- AI Agent
- Tool
- 환경 변수(.env)
- 설정 파일
- 실행 파일

프로젝트 단위로 작업하면 여러 개의 AI Agent를 서로 독립적으로 개발하고 관리할 수 있습니다.

2.6. 프로젝트 폴더 생성

이제 Google ADK 프로젝트를 생성해 보겠습니다.

먼저 프로젝트를 저장할 작업 폴더를 선택합니다.

예를 들어 다음과 같은 위치를 사용할 수 있습니다.

Windows

```
C:\workspace
```

작업 위치는 자유롭게 선택하면 됩니다.

2.7. 프로젝트 생성

명령 프롬프트를 실행한 후 작업 폴더를 만들고 이동합니다. 폴더가 이미 있으면 첫 줄에서 "이미 있습니다"가 표시되며, 그대로 진행하면 됩니다.

Windows (명령 프롬프트)

```
mkdir C:\workspace
cd C:\workspace
```

> 참고(macOS에서 재현할 때): `mkdir -p ~/workspace` 후 `cd ~/workspace`

프로젝트 폴더를 생성합니다.

```
mkdir my-adk
```

생성한 프로젝트 폴더로 이동합니다.

```
cd my-adk
```

현재 프로젝트 구조는 다음과 같습니다.

```
my-adk/
```

아직은 아무 파일도 없는 빈 프로젝트입니다.

2.8. VS Code에서 프로젝트 열기

생성한 프로젝트를 VS Code에서 엽니다.

메뉴를 이용하는 경우

```
File

↓

Open Folder

↓

my-adk 선택
```

또는 터미널에서 다음 명령을 사용할 수도 있습니다.

```
code .
```

프로젝트가 정상적으로 열리면 VS Code의 Explorer에 다음과 같이 표시됩니다.

```
my-adk
```

이제 Google ADK 프로젝트를 개발할 준비가 완료되었습니다.

2.9. 현재 프로젝트 구조

현재 프로젝트는 다음과 같은 구조입니다.

```
my-adk/
```

이후 실습을 진행하면서 다양한 파일과 폴더가 추가됩니다.

예를 들어 최종적으로는 다음과 같은 구조를 갖게 됩니다.

```
my-adk/

├── agent/
├── tools/
├── .env
├── requirements.txt
└── venv/
```

현재는 빈 프로젝트이지만 앞으로 AI Agent 개발에 필요한 코드와 설정 파일이 차례대로 추가될 예정입니다.

2.10. 가상환경 생성

Python 프로젝트는 사용하는 라이브러리의 버전이 서로 다를 수 있습니다.

예를 들어 프로젝트 A는

```
google-adk 1.0
```

을 사용하고,

프로젝트 B는

```
google-adk 2.0
```

을 사용할 수도 있습니다.

모든 프로젝트가 동일한 Python 환경을 사용하면 라이브러리 버전이 충돌할 수 있습니다.

이러한 문제를 해결하기 위해 **가상환경(Virtual Environment)** 을 사용합니다.

VS Code 메뉴에서 **Terminal** → **New Terminal**(단축키 `Ctrl + ``)을 선택해 터미널을 엽니다. 이 과정의 모든 명령은 **명령 프롬프트(cmd)** 기준이므로, 먼저 VS Code의 기본 터미널을 명령 프롬프트로 바꿉니다.

1. 터미널 패널 오른쪽 위 **+** 옆의 ∨ 버튼 클릭 → **Select Default Profile** 선택
2. 목록에서 **Command Prompt** 선택
3. 열려 있던 터미널을 휴지통 아이콘으로 닫고 **Terminal** → **New Terminal**로 다시 엽니다
4. 프롬프트가 `PS C:\...>`가 아니라 `C:\workspace\my-adk>` 형태로 표시되면 명령 프롬프트입니다

> **참고**
> VS Code의 기본 터미널인 PowerShell에서는 Windows 보안 정책(실행 정책) 때문에 가상환경 활성화 스크립트와 `npx` 명령이 차단되는 경우가 있습니다. 명령 프롬프트에서는 이 문제가 발생하지 않습니다.

명령 프롬프트에서 아래 명령을 실행합니다.

Windows (명령 프롬프트)

```
python -m venv venv
```

> 참고(macOS에서 재현할 때): `python3 -m venv venv`

명령이 완료되면 프로젝트 폴더 안에 **venv** 디렉터리가 생성됩니다.

```
my-adk/
├── venv/
```

이 디렉터리에는 프로젝트 전용 Python 실행 환경과 설치된 라이브러리가 저장됩니다.

2.11. 가상환경 활성화

가상환경을 생성한 후에는 반드시 활성화해야 합니다.

Windows (명령 프롬프트)

```
venv\Scripts\activate
```

> 참고(macOS에서 재현할 때): `source venv/bin/activate`

정상적으로 활성화되면 터미널 앞에 다음과 같이 표시됩니다.

```
(venv)
```

예를 들어

```
(venv) C:\workspace\my-adk>
```

와 같이 표시됩니다.

이는 현재 프로젝트 전용 Python 환경을 사용하고 있다는 의미입니다.

> **참고**
> 가상환경 활성화는 터미널 창 단위로 적용됩니다. VS Code를 다시 실행하거나 터미널을 새로 열 때마다 `my-adk` 폴더에서 `venv\Scripts\activate`를 다시 실행합니다. `(venv)` 표시가 없으면 이후의 `pip`, `adk` 명령이 동작하지 않습니다.

2.12. 왜 활성화해야 할까?

가상환경을 활성화하지 않으면 Python은 시스템에 설치된 전역(Global) 환경을 사용합니다.

이 경우 프로젝트별 라이브러리 관리가 어려워지고, 다른 프로젝트와 충돌이 발생할 수 있습니다.

따라서 Python 프로젝트를 시작할 때는 항상 다음 순서를 기억하는 것이 좋습니다.

```
프로젝트 열기

↓

가상환경 활성화

↓

라이브러리 설치
```

```
↓

개발 시작
```

정리

이번 절에서는 Google ADK 개발을 위한 Python 개발 환경을 구성했습니다.

항목	설명
Python	Google ADK가 동작하는 프로그래밍 언어
VS Code	Python 개발을 위한 코드 편집기
프로젝트 폴더	실습 코드를 저장하는 작업 공간
가상환경	프로젝트별 독립적인 Python 실행 환경
활성화	프로젝트 전용 Python 환경 사용

3. Google ADK 설치

3.1. Google ADK 설치

앞 절에서는 Python 개발 환경과 가상환경을 준비했습니다.

이제 Google ADK를 설치하여 AI Agent를 개발할 수 있는 환경을 구성합니다.

Google ADK는 Python 패키지 형태로 제공되며 `pip`(Python 패키지 설치 도구) 명령어를 이용하여 설치합니다.

먼저 가상환경이 활성화되어 있는지 확인합니다.

```
(venv)
```

가 표시된다면 다음 명령어를 실행합니다.

```
pip install google-adk
```

설치가 완료되면 Google ADK와 함께 필요한 의존성 라이브러리도 자동으로 설치됩니다.

3.2. 프로젝트 구조 만들기

Google ADK 프로젝트는 처음부터 폴더를 구분해 두는 것이 좋습니다.

이번 과정에서는 다음과 같은 구조를 사용합니다.

```
my-adk/
│
├── .env                    # API Key 저장
├── requirements.txt        # Python 패키지 목록
├── main.py                 # 프로그램 시작 파일
│
├── agent/
│   ├── __init__.py
│   └── agent.py       # 메인 Agent
│
├── tools/
│   ├── __init__.py
│   └── weather_tool.py     # Tool 예제
│
└── README.md
```

현재는 대부분의 파일이 비어 있지만, 강의가 진행되면서 하나씩 구현해 나갈 예정입니다.

3.3. 프로젝트 구조 설명

main.py

프로그램이 처음 실행되는 파일입니다.

사용자의 요청을 받아 Agent를 실행하는 역할을 합니다.

```
사용자

↓

main.py

↓

Agent 실행

↓

응답 출력
```

---

agent/

AI Agent를 관리하는 디렉터리입니다.

현재는 하나의 Agent만 사용하지만,

추후에는 여러 Agent를 추가할 예정입니다.

```
agent/

└── agent.py
```

---

tools/

AI가 사용할 Tool을 관리하는 디렉터리입니다.

예를 들어

```
날씨 조회

계산기

DB 조회

이메일 발송
```

등을 각각 하나의 Python 파일로 관리합니다.

이번 장에서는 Tool을 사용하지 않지만,

이후 실습에서 하나씩 추가할 예정입니다.

.env

API Key와 같은 민감한 정보를 저장합니다.

예를 들어

```
GOOGLE_API_KEY=xxxxxxxxxxxxxxxx
```

실무에서는 API Key를 Python 코드에 직접 작성하지 않습니다.

---

requirements.txt

프로젝트에서 사용하는 Python 패키지를 관리합니다.

예를 들어

```
google-adk
python-dotenv
```

처럼 필요한 라이브러리를 작성합니다.

README.md

프로젝트 설명서를 작성하는 파일입니다.

프로젝트 소개

설치 방법

실행 방법

등을 기록합니다.

GitHub에 프로젝트를 업로드할 때 가장 먼저 보게 되는 문서입니다.

3.4. requirements.txt 작성

프로젝트 루트에 다음 파일을 생성합니다.

```
requirements.txt
```

다음 내용을 작성합니다.

```
google-adk
python-dotenv
```

또는 버전을 함께 관리할 수도 있습니다.

```
google-adk==2.5.0
python-dotenv==1.1.0
```

> **참고**
>
> 버전 번호는 예시입니다. `pip show google-adk` 명령으로 현재 설치된 버전을 확인할 수 있습니다.

이후 다음 명령어를 실행하면 필요한 라이브러리를 한 번에 설치할 수 있습니다.

```
pip install -r requirements.txt
```

3.5. .env 파일 생성

프로젝트 루트에

```
.env
```

파일을 생성합니다.

Google AI Studio에서 발급받은 API Key를 저장합니다.

```
GOOGLE_API_KEY=여기에_API_KEY_입력
```

> **주의**
>
> `.env` 파일은 API Key와 같은 민감한 정보를 포함하므로 GitHub에 업로드하지 않아야 합니다. 일반적으로 `.gitignore` 파일에 `.env`를 추가하여 버전 관리 대상에서 제외합니다.

3.6. 설치 확인

Google ADK가 정상적으로 설치되었는지 확인합니다.

```
pip show google-adk
```

또는

```
pip list
```

를 실행하여

```
google-adk
```

가 포함되어 있는지 확인합니다.

정리

이번 절에서는 Google ADK를 설치하고 앞으로 사용할 프로젝트 구조를 구성했습니다.

구성 요소	역할
`main.py`	프로그램 시작점
`agent/`	AI Agent 관리
`tools/`	Tool 관리
`.env`	API Key 및 환경 변수
`requirements.txt`	프로젝트 의존성 관리
`README.md`	프로젝트 설명 문서

---

4. Gemini API Key 설정

4.1. API Key란?

Google ADK는 Gemini 모델과 통신하기 위해 Google의 AI 서비스를 사용합니다.

이때 Google 서버는

> "이 요청이 누구의 요청인지"

를 확인해야 합니다.

이를 위해 사용하는 것이 **API Key**입니다.

API Key는 서비스를 사용할 수 있도록 발급되는 인증 키입니다.

예를 들어

```
사용자

↓

Google ADK

↓

Gemini API

↓

API Key 확인

↓
```

```
응답
```

API Key가 올바르지 않으면 Gemini 모델을 사용할 수 없습니다.

4.2. Google AI Studio 접속 및 API KEY 발급

웹 브라우저에서 **Google AI Studio(**https://aistudio.google.com/)에 접속합니다.

Google 계정으로 로그인합니다.

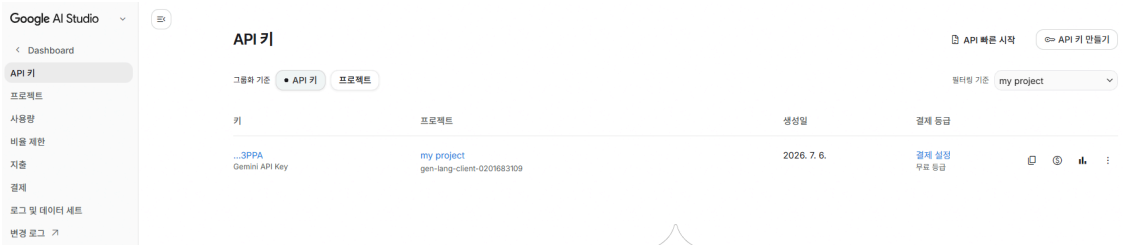
manage > Dashboard > 프로젝트로 이동합니다.

새 프로젝트 만들기를 클릭하고 [my project]라고 이름 입력 후, 프로젝트 만들기를 클릭합니다. API 키 발급을 위해서는 프로젝트가 생성되어야 합니다.

프로젝트 생성 이후에 새 키 만들기 화면이 활성화됩니다.

프로젝트를 선택하고 새 키 만들기를 클릭합니다.

아래와 같이 API 키 페이지에 새로운 키가 생성됩니다.



이후 키 이름을 클릭하여 세부 정보를 확인하고, 상단에 API 키를 복사합니다.

```
AIzaSy**************************
```

와 같은 문자열 형태로 구성되어 있습니다.

> **주의**
>
> 실제 API Key는 계정마다 다르며, 다른 사람과 공유해서는 안 됩니다.

4.3. 프로젝트에 API Key 등록

프로젝트 루트에 `.env` 파일을 생성합니다.

현재 프로젝트 구조는 다음과 같습니다.

```
my-adk/
│
├── .env
├── requirements.txt
├── main.py
│
├── agent/
└── tools/
```

`.env` 파일을 열고 다음과 같이 작성합니다.

```
GOOGLE_API_KEY=AIzaSy****************************
```

API Key는 큰따옴표(`"`) 없이 입력합니다.

4.4. python-dotenv 설치

`.env` 파일의 내용을 Python에서 읽기 위해 `python-dotenv` 라이브러리를 사용합니다.

다음 명령어를 실행합니다.

```
pip install python-dotenv
```

또는 `requirements.txt`에 추가한 후

```
pip install -r requirements.txt
```

를 실행합니다.

4.5. 환경 변수 불러오기

Python에서는 `python-dotenv`를 이용하여 `.env` 파일을 읽을 수 있습니다.

예를 들어 `main.py`에서 다음과 같이 작성합니다.

```
from dotenv import load_dotenv
import os

load_dotenv()

if os.getenv("GOOGLE_API_KEY"):
```

```
    print("API Key 로드 완료")
else:
    print("API Key를 찾을 수 없습니다. .env 파일을 확인하세요.")
```

이 코드를 실행하면 `.env` 파일의 환경 변수가 프로그램에 로드됩니다. API Key 값 자체는 화면에 출력하지 않고 존재 여부만 확인합니다.

이후 Google ADK는 `GOOGLE_API_KEY` 환경 변수를 사용하여 Gemini API와 통신합니다.

4.6. API Key 보안

API Key는 비밀번호와 같은 중요한 정보입니다.

다음 사항을 반드시 지켜야 합니다.

- 코드에 직접 작성하지 않습니다.
- GitHub에 업로드하지 않습니다.
- 다른 사람과 공유하지 않습니다.
- 공개 저장소에 포함하지 않습니다.

예를 들어 다음과 같은 코드는 좋은 방법이 아닙니다.

```
API_KEY = "AIzaSyXXXXXXXXXXXXXXXX"
```

API Key가 코드에 포함되면 외부에 노출될 위험이 있습니다.

대신 `.env` 파일을 사용하는 것이 안전합니다.

---

5. 첫 번째 Agent 실행

5.1. AI Agent 생성하기

이제 Google ADK를 이용하여 첫 번째 AI Agent를 만들어 보겠습니다.

이번 예제에서는 Tool을 사용하지 않습니다.

가장 기본적인 형태의 Agent를 생성하여 Google ADK의 동작 방식을 이해하는 것이 목표입니다.

현재 프로젝트 구조는 다음과 같습니다.

```
my-adk/
│
├── .env
├── main.py
│
├── agent/
│   ├── __init__.py
│   └── agent.py
│
└── tools/
```

이번 절에서는

```
agent.py
```

파일을 작성합니다.

`agent/agent.py`

```python
from google.adk.agents import Agent

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="hello_agent",
    description="첫 번째 AI Agent",
    instruction="""
    당신은 친절한 AI 비서입니다.

    사용자의 질문에 이해하기 쉽게 답변하세요.
    """
)
```

`agent/__init__.py`에는 다음 한 줄을 작성합니다. 이 파일이 있어야 `agent` 폴더가 Python 패키지로 인식되고, `adk web`이 폴더 안의 `root_agent`를 찾을 수 있습니다. Chapter 6, 7에서 만드는 Agent 폴더도 같은 방식으로 작성합니다.

`agent/__init__.py`

```python
from . import agent
```

Agent를 생성하는 코드는 생각보다 매우 간단합니다.

Google ADK에서는 Agent 객체 하나만 생성하면 AI Agent가 준비됩니다.

5.2. Agent 구성 요소

위 코드에는 여러 설정이 포함되어 있습니다.

```
Agent(
    ...
)
```

Agent 객체를 생성합니다.

```
model="gemini-3.1-flash-lite"
```

사용할 LLM을 지정합니다.

이번 과정에서는 가벼우면서 빠른 Gemini 3.1 Flash-Lite 모델을 사용합니다.

```
name="hello_agent"
```

Agent 이름입니다.

여러 Agent를 사용할 때 구분하기 위해 사용합니다.

```
description="첫 번째 AI Agent"
```

Agent의 설명입니다.

Multi Agent 환경에서 Agent의 역할을 설명할 때 사용됩니다.

```
instruction="""
...
"""
```

Agent의 역할을 정의합니다.

Instruction은 Agent의 성격과 행동 방식을 결정하는 가장 중요한 설정입니다.

5.3. Instruction 이해하기

Instruction은 AI에게

> "너는 어떤 역할을 수행해야 하는가"

를 알려주는 Prompt입니다.

예를 들어

```
당신은 친절한 AI 비서입니다.
```

라고 작성하면

AI는

친절한 말투로 답변하려고 합니다.

다음과 같이 변경해 봅시다.

```
당신은 Python 강사입니다.

항상 예제를 포함하여 설명하세요.
```

또는

```
당신은 여행 전문가입니다.

국내 여행만 추천하세요.
```

Instruction만 변경해도

동일한 Gemini 모델이

전혀 다른 Agent처럼 동작합니다.

5.4. Agent 실행

Agent를 실행하기 위해 별도의 실행 코드를 작성할 필요는 없습니다.

Google ADK의 CLI(Command Line Interface, 터미널에서 입력하는 명령어)가 `agent/` 폴더 안의 `root_agent`를 자동으로 찾아 실행합니다.

터미널에서 프로젝트 루트(`my-adk/`)로 이동한 후 다음 명령어를 실행합니다.

```
adk web
```

실행이 완료되면 브라우저에서 아래 주소로 접속합니다. `127.0.0.1`은 "내 컴퓨터 자신"을 뜻하는 주소(localhost)이고, `8000`은 그 안에서 `adk web` 프로그램을 구분하는 포트(Port) 번호입니다. 다른 PC에서는 접속할 수 없는 개발용 주소입니다.

http://127.0.0.1:8000/dev-ui/

화면 왼쪽 상단의 Agent 목록에서 `agent`를 선택하면 대화를 시작할 수 있습니다.

웹 UI 없이 터미널에서 바로 대화하려면 다음 명령어를 사용합니다.

```
adk run agent
```

> **참고**
>
> `adk run` 뒤의 `agent`는 Agent 코드가 들어 있는 폴더 이름입니다. 두 명령어 모두 프로젝트 루트에서 실행해야 하며, 이번 장에서는 `main.py`를 사용하지 않습니다.

5.5. 실행 과정

Agent는 다음과 같은 순서로 동작합니다.

```
사용자 질문

↓

Agent

↓

Gemini

↓

응답 생성

↓

사용자
```

현재는 Tool이 없기 때문에

Gemini가 직접 응답을 생성합니다.

5.6. 첫 번째 질문

Agent가 실행되면

다음과 같이 질문해 봅니다.

```
안녕하세요.
```

↓

응답

```
안녕하세요.

무엇을 도와드릴까요?
```

또는

```
Python이 무엇인가요?
```

↓

Gemini

↓

설명 생성

↓

응답

5.7. Instruction 변경해보기

이번에는

Instruction를 변경합니다.

예를 들어

```
당신은 유치원 선생님입니다.

항상 어린이가 이해할 수 있도록 설명하세요.
```

다시 실행한 후

같은 질문을 입력합니다.

```
Python이 무엇인가요?
```

응답이 이전과 달라지는 것을 확인할 수 있습니다.

또 다른 예입니다.

```
당신은 20년 경력의 Java 개발자입니다.

실무 중심으로 설명하세요.
```

같은 질문을 입력해도 응답 스타일이 달라집니다.

이처럼 Instruction은 Agent의 역할을 결정하는 핵심 요소입니다.

5.8. 현재 Agent의 한계

현재 Agent는 질문에는 답변할 수 있지만

다음과 같은 작업은 수행할 수 없습니다.

- 현재 시간 조회
- 날씨 조회
- 데이터베이스 조회
- 이메일 발송
- 계산기 사용

왜일까요?

현재 Agent에는 Tool이 연결되어 있지 않기 때문입니다.

즉,

```
사용자

↓

Agent

↓
```

```
Gemini

↓

응답
```

구조만 존재합니다.

실습 과제

실습 2-1. 첫 번째 질문

다음 질문을 입력해 봅니다.

- 자기소개를 해주세요.

실습 2-2. Instruction 역할 변경

Instruction를 다음과 같이 변경하여 응답을 비교해 보세요.

- Python 강사
- 여행 전문가
- 고객 상담원

실습 2-3. Instruction 조건 추가

Instruction에 다음 조건을 추가해 보세요.

- 항상 3줄 이내로 답변
- 항상 예제를 포함
- 항상 존댓말 사용

응답이 어떻게 달라지는지 확인합니다.

정리

이번 절에서는 Google ADK를 이용하여 첫 번째 AI Agent를 생성하고 실행했습니다.

구성 요소	역할
Agent	AI의 역할과 동작을 정의하는 객체

구성 요소	역할
Model	사용할 LLM 지정
Name	Agent 식별자
Description	Agent 설명
Instruction	AI의 역할과 행동을 정의하는 프롬프트

최종 정리

이번 장에서는 Google ADK 개발을 위한 환경을 구축하고 첫 번째 Agent를 실행했습니다.

핵심 내용은 다음과 같습니다.

개념	핵심 내용
Python 가상환경	프로젝트별 독립된 실행 환경
Google ADK	AI Agent 개발 프레임워크
API Key	Gemini 모델 인증 정보
Agent 실행	ADK를 이용해 첫 번째 Agent 실행

> **참고**
> 이번 장에서 만든 `my-adk` 프로젝트는 1일차 전체에서 계속 사용합니다. 이후 챕터의 Tool, Agent, MCP 서버는 모두 이 프로젝트 안에 폴더를 추가하는 방식으로 작성하며, 가상환경과 API Key 설정을 다시 하지 않습니다.

체크포인트 질문

1. 가상환경을 활성화하지 않고 `pip install google-adk`를 실행하면 어떤 문제가 생길 수 있나요? → **전역 Python 환경에 설치되어 다른 프로젝트와 라이브러리 버전이 충돌할 수 있음**
2. API Key를 `.env` 파일에 저장하고 코드에 직접 쓰지 않는 이유는 무엇인가요? → **코드가 공유되거나 GitHub에 올라갈 때 API Key가 노출되는 것을 막기 위해**
3. `adk web` 실행 후 브라우저의 Agent 목록에 표시되는 이름은 `name="hello_agent"`일까요, 폴더명 `agent`일까요? → **폴더명 `agent`. `name`은 코드 안에서 Agent를 구분하는 값**
4. 같은 Gemini 모델을 사용하는데 응답 스타일이 달라지게 만드는 설정은 무엇인가요? → **Instruction**